Università di Genova

UNIVERSITY OF GENOA
Master's Degree in Robotics Engineering (Class LM-32)
Program Code: 11963
Academic Year: 2025/2026

Lab 2 Delivery: ROV Controllability Analysis

CONTROL OF MULTI-VARIABLE SYSTEMS – CODE: 80181

Student: HUSSEIN MOHAMED ABDELAZIM MOHAMED MOSTAFA ELNAGGAR

Student ID: 8786438

Email: S8786438@studenti.unige.it || eng.h.elnagar@gmail.com

November 13, 2025

Table of Contents

1. Introduction 2
2. Define the System on MATLAB 2
2.1 State-Space Model 2
2.2 State Matrix (A) 2
2.3 Input Matrix (B) 3
3. Check System Controllability 4
3.1 Theory 4
3.2 Implementation 4
3.3 Results and Conclusion 4
4. Compute the System Input 5
4.1 Theory 5
4.2 Implementation 5
5. Simulate the System 6
5.1 Theory 6
5.2 Implementation 6
6. Plot the State Evolution 6
6.1 Results 6
6.2 Analysis and Conclusion 7
7. Controllability Analysis with Thruster Failures 8
7.1 Analysis of Thruster 2 is Not Working 8
7.1.1 Analysis of Thruster 8
7.1.2 Results and Conclusion 8
7.2 Repeat by Disabling Thrusters 2 and 3 9
7.2.1 Analysis of thrusters 9
7.2.2 Results and Conclusion 9
8. Final Lab Summary 10

1. Introduction

The primary objectives of this lab are:

- To model the 6-state dynamic system of the ROV in MATLAB using a state-space representation.
- To analyze the controllability of this system by computing the Reachability Gramian matrix.
- To compute the necessary control input u(t) to steer the system from an initial state $x_0 = [0]$ to a desired final state x_f in a given time T.
- To simulate the system's evolution with this input and verify that it reaches the target state.
- To analyze the system's robustness by investigating how thruster failures affect its controllability.

2. Define the System on MATLAB

2.1 State-Space Model

The system is modeled as a linear time-invariant (LTI) system:

$$\dot{x} = Ax + Bu$$

Based on the problem description and the plots, the 6-dimensional state vector x is defined as:

$$x = [x, y, \theta, \dot{x}, \dot{y}, \dot{w}]^T$$

Where:

- x: Position in the X-direction (Surge)
- y: Position in the Y-direction (Sway)
- θ: Orientation angle (Yaw)
- $\dot{x}$: Velocity in X (v_x)
- $\dot{y}$: Velocity in Y (v_y)
- $\dot{w}$: Angular velocity (Yaw rate)

The 4-dimensional input vector u consists of the voltages applied to the four thrusters:

$$u = [V_1, V_2, V_3, V_4]^T$$

2.2 State Matrix (A)

The state matrix A is derived from the ROV's dynamic equations. The upper-right quadrant is an identity matrix representing the kinematic relationship ($\dot{p} = v$), and the lower-right quadrant contains the damping terms (-alfa/m, -beta/m, -gamma/J).

MATLAB Implementation: The following code defines the physical parameters and constructs the A matrix.

```
%% Define matrices
alfa = 1.4;
beta = 1.8;
gamma = 1.05;
m = 13.5;
J = 0.37;
% State Matrix A (6x6)
A = [0 0 0 1 0 0;
    0 0 0 0 1 0;
    0 0 0 0 0 1;
    0 0 0 -alfa/m 0 0;
    0 0 0 0 -beta/m 0;
    0 0 0 0 0 -gamma/J];
```

2.3 Input Matrix (B)

The input matrix B maps the thruster voltages u to the state derivatives $\dot{x}$. This mapping is a two-step process:

1- The voltages u are converted to forces $\tau = [F_x, F_y, N_z]^T$ via the Thruster Allocation Matrix (TAM) and a DC gain K.

$$\tau = K \cdot \text{TAM} \cdot u$$

2- These forces are then mapped to accelerations (the derivatives of velocity states) via the mass m and inertia J.

This entire chain is consolidated into the single B matrix, where:

$$B = B_{\text{force_gain}} \cdot K \cdot \text{TAM}.$$

MATLAB Implementation: The TAM is defined based on the 3-DOF model for 4 thrusters. The B matrix is then constructed. With A and B defined, the system model is complete.

```
% Thruster Allocation Matrix (3x4)
TAM = [0.7071 0.7071 -0.7071 -0.7071;    % Surge
      -0.7071 0.7071 -0.7071 0.7071;    % Sway
      -0.1888 0.1888 0.1888 -0.1888];   % Yaw
% Input Matrix B (6x4)
B_force_gain = [zeros(3,3);
                1/m 0 0;
                0 1/m 0;
                0 0 1/J];
K_gain = 30; % DC-gain from volts to force
B = B_force_gain * K_gain * TAM;
```

3. Check System Controllability

3.1 Theory

To determine if the system is controllable (or, more precisely, reachable), we use the Reachability Gramian Theorem. The theorem states that an LTI system is completely reachable in time T if and only if the Reachability Gramian G(0,T) has full rank.

The Gramian is defined by the integral:

$$G(0,T) = \int_0^T e^{A(T-\tau)} BB^\top e^{A^\top (T-\tau)} d\tau$$

For our 6-state system, "full rank" means Rank(G) = 6.

3.2 Implementation

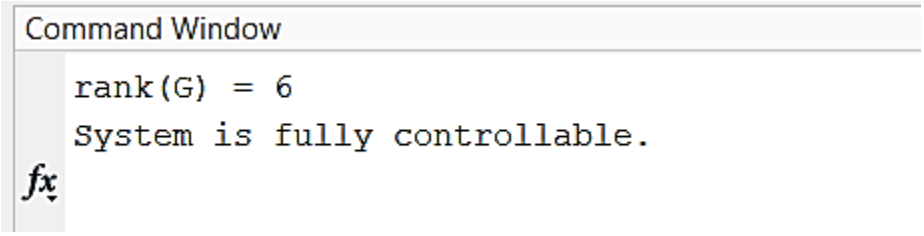
We use MATLAB's Symbolic Math Toolbox to define the variable of integration tau, compute the matrix exponential eA, define the integrand dG, and finally compute the definite integral for G from 0 to T=20. We then find its rank.

MATLAB Implementation:

```
T = 20;
dt = 0.01;
xf = [-1 2 3 0 0 0]';
syms tau real
eA = expm(A*(T-tau)); % Symbolic matrix exponential
dG = eA * B * B' * eA'; % Gramian integrand
G = eval(int(dG,tau,0,T));
rankG = rank(G);
disp(['rank(G) = ' num2str(rankG)]);
```

3.3 Results and Conclusion

Upon running the code, the following output was generated in the Command Window:



Conclusion: The rank of the Gramian matrix G is 6. This is equal to the number of states in the system. Therefore, the system is fully controllable. This confirms that it is theoretically possible to compute an input u(t) that can steer the ROV from the origin to any desired final state x_f in 20 seconds.

4. Compute the System Input

4.1 Theory

Since the system is confirmed to be controllable, we can compute a control input u(t) that drives the system from $x_0 = [0]$ to $x_f = [-1,2,3,0,0,0]^T$ in $T = 20$ seconds.

The standard formula for the minimum-energy control input that achieves this transfer is given by:

$$u(t) = B^T e^{A^T(T-t)} G(0,T)^{-1} x_f$$

This formula uses the already-computed Gramian inverse (G^{-1}), the state matrix (A), the input matrix (B), and the target state (x_f).

4.2 Implementation

We implement this formula directly in MATLAB. A key challenge is converting this symbolic function of time into a numerical matrix that can be used in the simulation loop.

Invert Gramian: We first compute the inverse of the Gramian, iG.

Define Symbolic Input: We define the symbolic function u(tau) using the formula.

Evaluate Numerically: We create a numerical time vector time with step dt. We then evaluate the symbolic function u at every point in this time vector.

Format uvalue: The output u_time is a cell array. We must re-stack the evaluated vectors vertically (using semicolons [;]) to create a single (4 x LL) numerical matrix uvalue, where each column represents the input vector $[V_1, V_2, V_3, V_4]^T$ at that specific time step.

MATLAB Implementation:

```
if rankG == 6 % Check for full rank
   disp('System is fully controllable.');
   %% Define time vector
   time = 0:dt:T;
   %time = time'; % This line is commented out
   LL = length(time);


   %% Compute system input
   iG = inv(G); % invert Gramian
   u(tau) = B'*eA'*iG*xf;
   u_time = u(time);

   % Stack the 4 row vectors vertically to create the 4xLL uvalue matrix
   uvalue = [eval(u_time{1}); eval(u_time{2}); eval(u_time{3}); eval(u_time{4})];
```

This uvalue matrix now contains the precise voltage commands for all four thrusters for the entire 20-second maneuver.

5. Simulate the System

5.1 Theory

With the input matrix uvalue computed, we can now simulate the system's response. We do this by numerically integrating the state equation $\dot{x} = Ax + Bu$ over time.

We initialize the state $x(1) = x_0 = [0]$. Then, for each time step k, we:

- Read the control input u_k from the uvalue matrix.
- Calculate the state derivative $\dot{x}(k) = Ax(k) + Bu_k$.
- Estimate the next state $x(k+1)$ using Euler integration:

$$x(k+1) = x(k) + \dot{x}(k) \cdot dt$$

5.2 Implementation

A for loop is used to iterate from the first time step (k=1) to the second-to-last (LL-1). Inside the loop, the Euler integration step is performed. A matrix x of size (6 x LL) is pre-allocated with zeros to store the state evolution at each time step.

MATLAB Implementation:

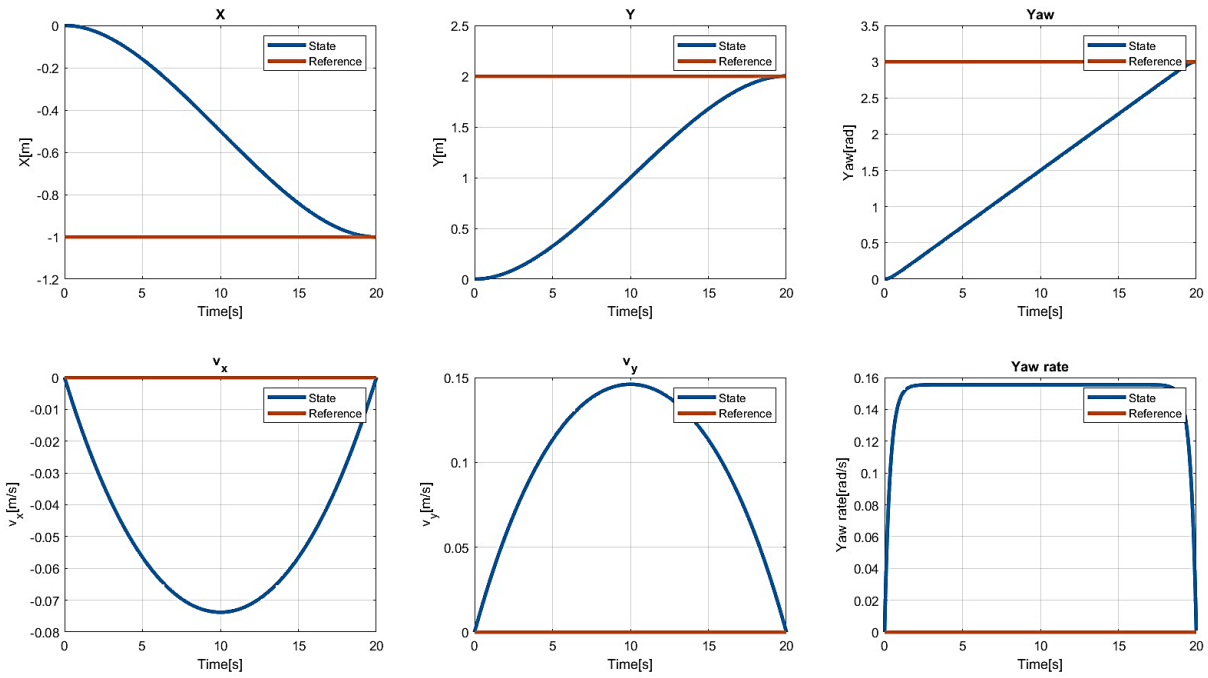
```
%% Initialize state
x0 = [0 0 0 0 0 0]'; % we start from the origin
x = zeros(6,LL); % this matrix will store the state evolution
x(:,1) = x0;
%% Simulate system (you can use Euler integration)
for k = 1:LL-1
    u_k = uvalue(:,k); % Input vector at time k (4x1)
    xdot = A * x(:,k) + B * u_k; % State derivative (6x1)
    x(:,k+1) = x(:,k) + xdot * dt; % Euler integration step
end
```

6. Plot the State Evolution

6.1 Results

After the simulation loop completes, the x matrix contains the full history of the ROV's state. The final part is dedicated to plotting this data against the desired reference state xf.

The code generates a figure with 6 subplots, one for each state. The blue line (State) is the simulated trajectory from the x matrix. The red line (Ref.) is the constant target value from xf.



Plot Output: The generated figure (Figure 1) visually confirms the result.

6.2 Analysis and Conclusion

The simulation results provide a clear validation of the entire process:

Final State Error: The final error 0.0024227 is extremely small. This numerical error is expected due to the nature of numerical integration (Euler's method) and floating-point precision. It confirms that the system successfully reached the target state.

Position States (X, Y, Yaw): The plots for X, Y, and Yaw (top row) show the blue "State" line starting at 0 and perfectly matching the red "Reference" line at $t = 20$ s. The ROV moved to $x = -1$ m, $y = 2$ m, and rotated to $\theta = 3$ rad.

Velocity States (v_x, v_y, Yaw rate): The plots for the velocities (bottom row) show the system starting and ending at zero velocity, as specified in x_f. The non-zero values in between represent the necessary motion to achieve the position change within 20 seconds.

Conclusion: The simulation and the plots successfully demonstrate that the control input computed was correct and effectively steered the controllable system from its initial state to the desired final state.

7. Controllability Analysis with Thruster Failures

This final section analyzes the system's robustness by assessing its controllability after one or more thruster failures. The system's ability to maintain controllability despite component failure is a key indicator of its redundancy.

7.1 Analysis of Thruster 2 is Not Working

7.1.1 Analysis of Thruster

To simulate the failure of Thruster 2, the second column of the Thruster Allocation Matrix (TAM) is set to a vector of zeros. This modification means that V_2 (voltage to thruster 2) no longer contributes any force or torque to the system.

A new input matrix B_mod1 and a new Gramian G_mod1 are calculated, and the rank is checked again.

MATLAB Implementation:

```
%% Analysis: Thruster 2 Fails
disp('--- Analysis 1.6: Thruster 2 Fails ---');
TAM_mod1 = TAM;
TAM_mod1(:, 2) = 0; % Thruster 2 (second column) fails
B_mod1 = B_force_gain * K_gain * TAM_mod1;


% We need eA and tau from the base script
dG_mod1 = eA * B_mod1 * B_mod1' * eA';
G_mod1 = eval(int(dG_mod1, tau, 0, T));
rankG_mod1 = rank(G_mod1);
disp(['Rank with Thruster 2 failure = ' num2str(rankG_mod1)]);


if rankG_mod1 == 6
   disp('System IS STILL fully controllable.');
else
   disp('System is NOT controllable.');
end
```

7.1.2 Results and Conclusion

The code execution produced the following output:

```
--- Analysis 1.6: Thruster 2 Fails ---
Rank with Thruster 2 failure = 6
System IS STILL fully controllable.
```

Conclusion: Even with the complete failure of Thruster 2, the system remains fully controllable. The rank of the Gramian is still 6. This demonstrates that the initial 4-thruster configuration has actuator redundancy. The remaining three thrusters (1, 3, and 4) are sufficient to generate any required combination of forces (Surge, Sway) and torque (Yaw) to control all 6 states.

7.2 Repeat by Disabling Thrusters 2 and 3

7.2.1 Analysis of thrusters

This scenario models a more severe failure, where both Thruster 2 and Thruster 3 are non-functional. To simulate this, both the second and third columns of the TAM are set to zeros, leaving only Thrusters 1 and 4 operational.

The process is repeated: a new B_mod2 and G_mod2 are calculated, and the rank is determined.

MATLAB Implementation:

```
%% Analysis 1.7: Thrusters 2 & 3 Fail
disp('--- Analysis 1.7: Thrusters 2 & 3 Fail ---');
TAM_mod2 = TAM;
TAM_mod2(:, [2, 3]) = 0; % Thrusters 2 and 3 fail


B_mod2 = B_force_gain * K_gain * TAM_mod2;
dG_mod2 = eA * B_mod2 * B_mod2' * eA';
G_mod2 = eval(int(dG_mod2, tau, 0, T));
rankG_mod2 = rank(G_mod2);
disp(['Rank with Thrusters 2 & 3 failure = ' num2str(rankG_mod2)]);


if rankG_mod2 < 6
   disp('System is NOT fully controllable.');
else
   disp('System IS STILL controllable.');
end
```

7.2.2 Results and Conclusion

The code execution produced the following output:

```
--- Analysis 1.7: Thrusters 2 & 3 Fail ---
Rank with Thrusters 2 & 3 failure = 5
System is NOT fully controllable.
>>
```

Conclusion: With the failure of two thrusters, the system is no longer fully controllable. The rank of the Gramian has dropped from 6 to 5. This means there is at least one dimension (or direction) in the 6-dimensional state space that the ROV can no longer reach or control.

The physical reason is that the system has lost its actuator redundancy. The remaining two thrusters (1 and 4) are not sufficient to independently generate arbitrary forces in all three 3-DOF directions (Surge, Sway, Yaw). This loss of a "degree of freedom" in actuation results in the loss of full controllability for the state-space model.

8. Final Lab Summary

- This lab successfully demonstrated the complete process of controllability analysis for an LTI system representing a 3-DOF ROV.
- The system was defined in MATLAB, and its controllability was confirmed (Rank = 6).
- A valid control input was computed to steer the system from $x_0 = [0]$ to a non-trivial target x_f.
- The simulation validated this input, showing the system reaching the target state with minimal numerical error.
- Finally, a robustness analysis demonstrated that the system was redundant to a single thruster failure but not to a double thruster failure, which resulted in a loss of controllability.